



AWS Cloud Development Kit (CDK)

- Define your cloud infrastructure using a familiar language:
 - JavaScript/TypeScript, Python, Java, and .NET
- Contains high level components called **constructs**
- The code is “compiled” into a CloudFormation template (JSON/YAML)
- You can therefore deploy infrastructure and application runtime code together
 - Great for Lambda functions
 - Great for Docker containers in ECS / EKS

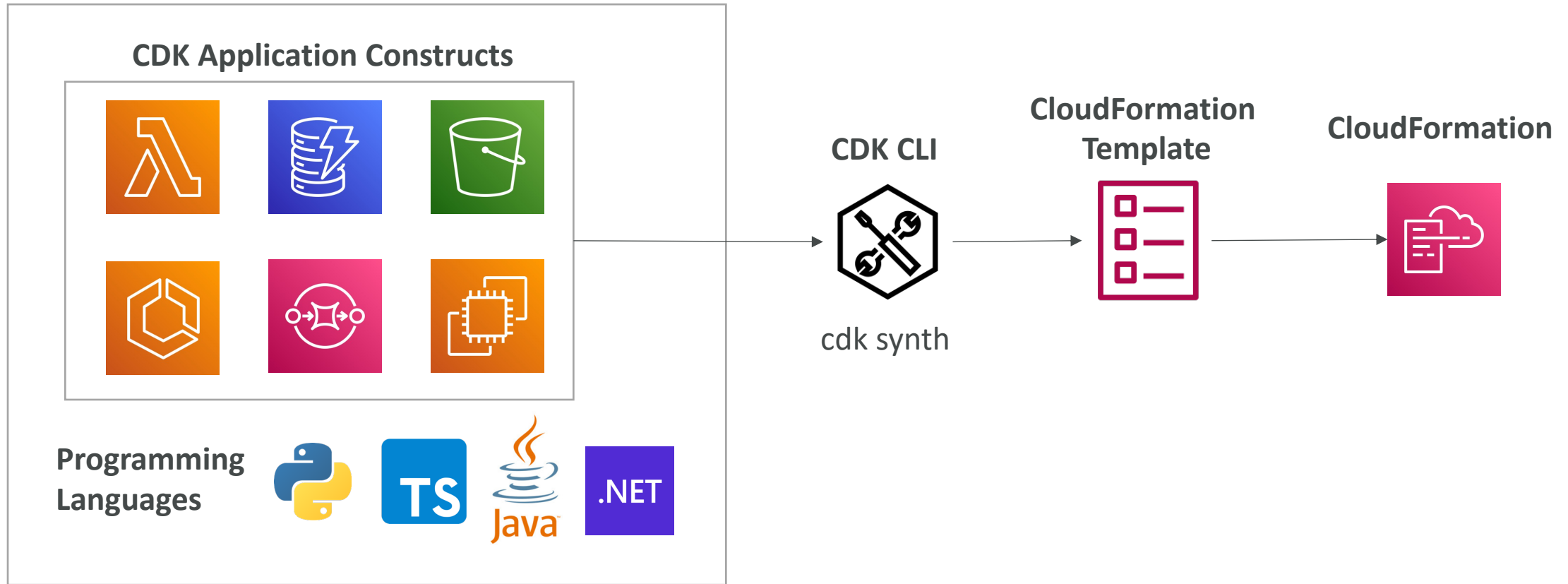
```
export class MyEcsConstructStack extends core.Stack {
  constructor(scope: core.App, id: string, props?: core.StackProps) {
    super(scope, id, props);

    const vpc = new ec2.Vpc(this, "MyVpc", {
      maxAzs: 1 // Default is all AZs in region
    });

    const cluster = new ecs.Cluster(this, "MyCluster", {
      vpc: vpc
    });

    // create a load-balanced Fargate service and make it public
    new ecs_patterns.ApplicationLoadBalancedFargateService(this, "My
      cluster: cluster, // Required
      cpu: 512, // Default is 256
      desiredCount: 6, // Default is 1
      taskImageOptions: { image: ecs.ContainerImage.fromRegistry("an
      memoryLimitMiB: 2048, // Default is 512
      publicLoadBalancer: true // Default is false
    });
  }
}
```

CDK in a diagram



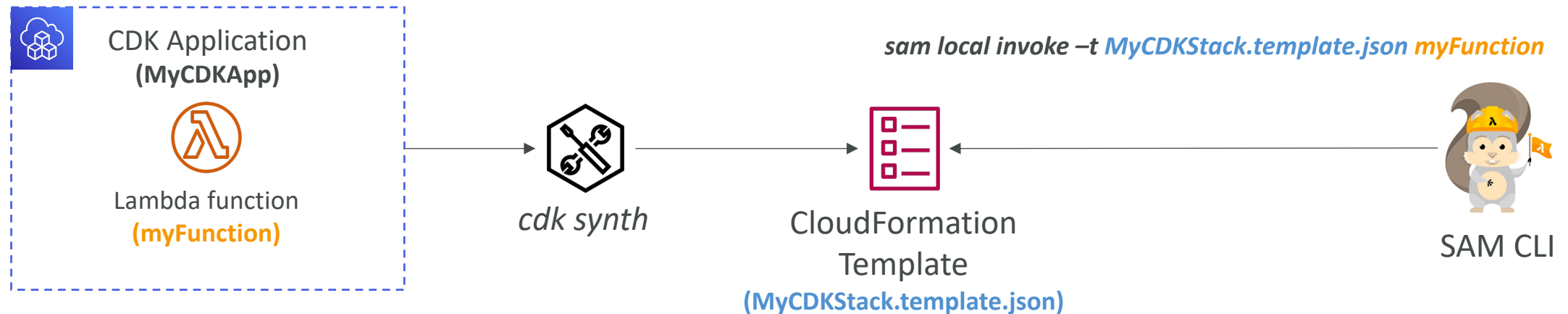
CDK vs SAM



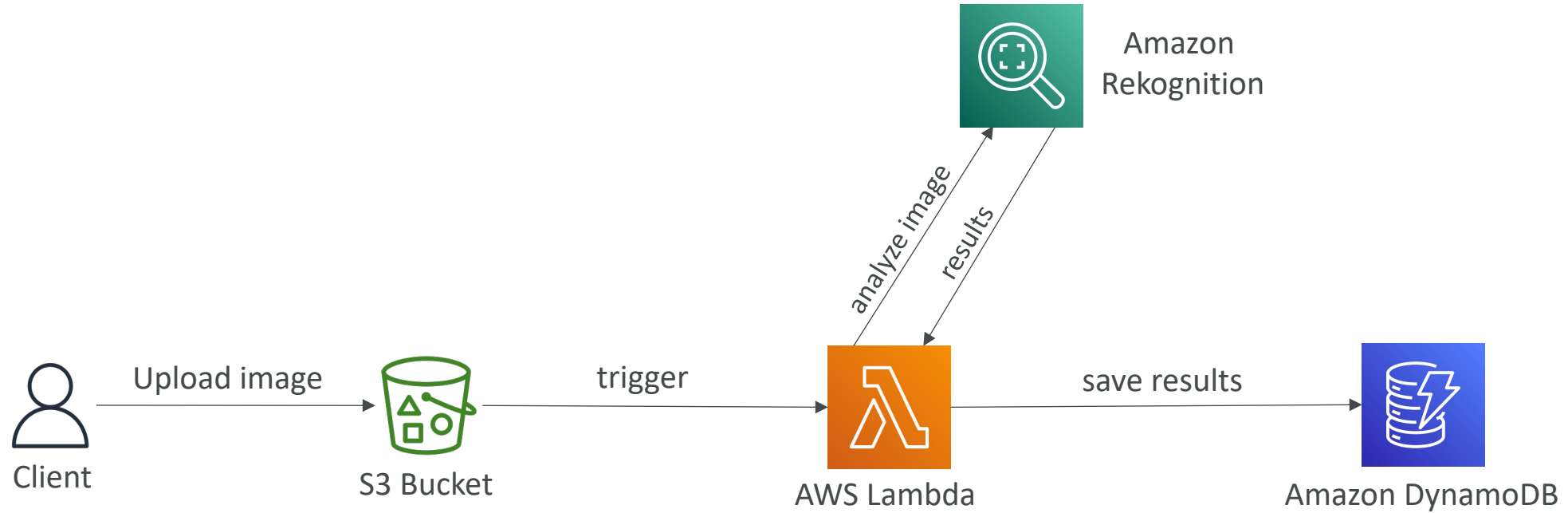
- SAM:
 - Serverless focused
 - Write your template declaratively in JSON or YAML
 - Great for quickly getting started with Lambda
 - Leverages CloudFormation
- CDK:
 - All AWS services
 - Write infra in a programming language JavaScript/TypeScript, Python, Java, and .NET
 - Leverages CloudFormation

CDK + SAM

- You can use SAM CLI to locally test your CDK apps
- You must first run *cdk synth*



CDK Hands-On



CDK Constructs

- CDK Construct is a component that encapsulates everything CDK needs to create the final CloudFormation stack
- Can represent a single AWS resource (e.g., S3 bucket) or multiple related resources (e.g., worker queue with compute)
- **AWS Construct Library**
 - A collection of Constructs included in AWS CDK which contains Constructs for every AWS resource
 - Contains 3 different levels of Constructs available (L1, L2, L3)
- **Construct Hub** – contains additional Constructs from AWS, 3rd parties, and open-source CDK community

CDK Constructs – Layer I Constructs (LI)

- Can be called *CFN Resources* which represents all resources directly available in CloudFormation
- Constructs are periodically generated from CloudFormation Resource Specification
- Construct names start with Cfn (e.g., CfnBucket)
- You must explicitly configure all resource properties

```
const bucket = new s3.CfnBucket(this, "MyBucket", {  
    bucketName: "MyBucket"  
});
```

CDK Constructs – Layer 2 Constructs (L2)

- Represents AWS resources but with a higher level (intent-based API)
- Similar functionality as L1 but with convenient defaults and boilerplate
 - You don't need to know all the details about the resource properties
- Provide methods that make it simpler to work with the resource (e.g., `bucket.addLifecycleRule()`)

```
const s3 = require('aws-cdk-lib/aws-s3');

const bucket = new s3.Bucket(this, 'MyBucket', {
  versioned: true,
  encryption: s3.BucketEncryption.KMS
});

// Returns the HTTPS URL of an S3 Object
const objectUrl = bucket.urlForObject('MyBucket/MyObject');
```


CDK Constructs – Layer 3 Constructs (L3)

- Can be called *Patterns*, which represents multiple related resources
- Helps you complete common tasks in AWS
- Examples:
 - `aws-apigateway.LambdaRestApi` represents an API Gateway backed by a Lambda function
 - `aws-ecs-patterns.ApplicationLoadBalancerFargateService` which represents an architecture that includes a Fargate cluster with Application Load Balancer

```
const api = new apigateway.LambdaRestApi(this, 'myapi', {
  handler: backend,
  proxy: false
});

const items = api.root.addResource('items');
items.addMethod('GET'); // GET /items
items.addMethod('POST'); // POST /items

const item = items.addResource('{item}');
item.addMethod('GET'); // GET /items/{item}

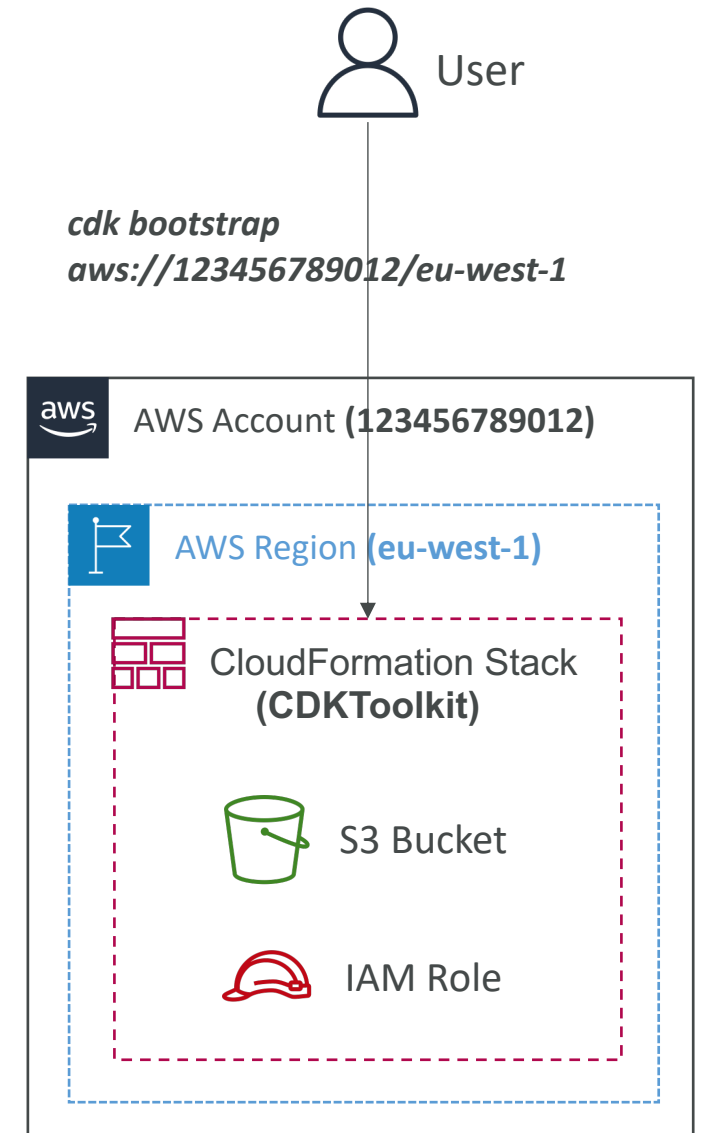
item.addMethod('DELETE', new apigateway.HttpIntegration('http://amazon.com'));
```

CDK – Important Commands to know

Command	Description
npm install -g aws-cdk-lib	Install the CDK CLI and libraries
cdk init app	Create a new CDK project from a specified template
cdk synth	Synthesizes and prints the CloudFormation template
cdk bootstrap	Deploys the CDK Toolkit staging Stack
cdk deploy	Deploy the Stack(s)
cdk diff	View differences of local CDK and deployed Stack
cdk destroy	Destroy the Stack(s)

CDK – Bootstrapping

- The process of provisioning resources for CDK before you can deploy CDK apps into an AWS environment
- **AWS Environment** = account & region
- CloudFormation Stack called **CDKToolkit** is created and contains:
 - **S3 Bucket** – to store files
 - **IAM Roles** – to grant permissions to perform deployments
- You must run the following command for each new environment:
 - `cdk bootstrap aws://<aws_account>/<aws_region>`
- Otherwise, you will get an error “Policy contains a statement with one or more invalid principal”



CDK – Testing

- To test CDK apps, use **CDK Assertions Module** combined with popular test frameworks such as Jest (JavaScript) or Pytest (Python)
- Verify we have specific resources, rules, conditions, parameters...
- Two types of tests:
 - **Fine-grained Assertions (common)** – test specific aspects of the CloudFormation template (e.g., check if a resource has this property with this value)
 - **Snapshot Tests** – test the synthesized CloudFormation template against a previously stored baseline template
- To import a template
 - `Template.fromStack(MyStack)` : stack built in CDK
 - `Template.fromString(mystring)` : stack build outside CDK

```
describe("StateMachineStack", () => {
  test("synthesizes the way we expect", () => {
    ...

    // Prepare the stack for assertions
    const template = Template.fromStack(MyStack);

    // Assert it creates Lambda with correct properties...
    template.hasResourceProperties("AWS::Lambda::Function", {
      Handler: "handler",
      Runtime: "nodejs14.x",
    });
    // Assert it creates the SNS subscription...
    template.resourceCountIs("AWS::SNS::Subscription", 1);

    // Assert the synthesized CloudFormation template
    // against a previously stored baseline template
    expect(template.toJSON()).toMatchSnapshot();
  });
});
```

Fine-grained Assertions

Snapshot Test